

Homework A3: Datapath, Pipelining, and Hazards

EECE.4821/5821 Computer Architecture and Design

Due: 10/07/2026

How to submit

This assignment has two Gradescope submissions.

1. **HWA3 – Autograded.** Download the starter files `answers.txt` and `schedule.s`. Put every **[Autograded]** answer next to its key in `answers.txt` (each part below shows its key, e.g. `Pla_lw`). Put your reordered code for Problem 4(c) in `schedule.s`. Upload *both* files.
2. **HWA3 – Written.** Upload one PDF with your answers to the **[Written]** parts (typed or neatly handwritten and scanned). Show your work there for any autograded part if you want the grader to see your reasoning.

Feedback. As soon as you submit, Gradescope runs a *format check* on `answers.txt` (it tells you about missing or unreadable answers, not whether they are right) and reports whether your `schedule.s` is correct and how many cycles it takes. You may resubmit as often as you like before the deadline; the scores for the other autograded answers are released after the due date.

Answer format. Numbers only; units are optional (320 and 320 ps are both accepted). Give non-integer answers to at least three significant figures.

Points. Undergraduate (EECE.4821): 100 points (84 autograded, 16 written). Graduate (EECE.5821): 120 points, adding the graduate problem.

Pipeline conventions used in every problem

- The pipeline is the 5-stage RISC-V pipeline from lecture: IF, ID, EX, MEM, WB. Instructions are issued in order, one per cycle, unless the pipeline stalls.
- The first instruction is fetched in **cycle 1**. The **total cycle count** of a code sequence is the cycle in which its last instruction is in WB.
- The register file is written in the first half of a cycle and read in the second half, so an instruction in ID can read a value written by WB in that same cycle.
- A stalled instruction waits in ID, and the instruction behind it waits in IF. A **stall** is one cycle in which a bubble is inserted.
- **Full forwarding** means the forwarding paths from lecture: from the EX/MEM pipeline register to the ALU inputs, and from the MEM/WB pipeline register to the ALU inputs. With full forwarding, the only data hazard that still stalls is a load followed immediately by an instruction that uses the loaded register (1 stall). A `sw` needs both of its source registers when it enters EX, just like every other instruction.

Problem 1: Single-cycle vs. pipelined performance

(20 pt)

A designer measures the latency of each section of a RISC-V datapath:

Stage	IF	ID	EX	MEM	WB
	instruction fetch	decode + register read	ALU	data memory	register write
Latency	250 ps	150 ps	200 ps	300 ps	100 ps

In the single-cycle design, an instruction's delay is the sum of the sections it uses: `lw` uses all five; `sw` uses all but WB; R-type uses all but MEM; `beq` uses only IF, ID, and EX. The program being studied has this instruction mix: `lw` 25%, `sw` 10%, R-type 45%, `beq` 20%.

- (a) [Autograded] What is the single-cycle delay of each instruction class, in ps? `P1a_lw` `P1a_sw` `P1a_rtype` `P1a_beq` (4 pt)
- (b) [Autograded] What is the clock period of the single-cycle design? `P1b_clock` (1 pt)
- (c) [Autograded] Suppose, hypothetically, that the clock period could change from instruction to instruction so that each instruction took only as long as it needed. What would the average time per instruction be for this mix? `P1c_avg` (2 pt)
- (d) [Autograded] The datapath is pipelined into the five stages above. Each pipeline register adds 20 ps of overhead to every stage. What is the clock period? `P1d_clock` (2 pt)
- (e) [Autograded] In the pipelined design, how long does it take for a single `lw` to go from the start of IF to the end of WB? `P1e_lw_latency` (2 pt)
- (f) [Autograded] For a very long program with no hazards (CPI = 1), what is the speedup of the pipelined design over the single-cycle design? `P1f_speedup` (2 pt)
- (g) [Autograded] The designer splits MEM into two stages of 150 ps each, making a 6-stage pipeline (still 20 ps of overhead per stage). Give the new clock period, the new speedup over the single-cycle design, and the stage that now limits the clock. `P1g_clock` `P1g_speedup` `P1g_bottleneck` (3 pt)
- (h) [Written] The pipeline in part (d) has five stages, but the speedup in part (f) is well below 5. In 2–4 sentences, give the reasons. Then name one more effect, not included in part (f), that would lower the speedup of a real program even further. (4 pt)

Problem 2: Reading a pipeline

(12 pt)

The following five instructions run on the 5-stage pipeline. They are independent, so there are no hazards and no stalls.

```
I1: lw    x9, 8(x20)
I2: or    x5, x6, x7
I3: sw    x12, 16(x21)
I4: and   x13, x14, x15
I5: sub   x16, x17, x18
```

It may help to sketch a multi-cycle pipeline diagram first. You do not need to submit it.

- (a) [Autograded] How many cycles does the sequence take (I1 fetched in cycle 1 through I5 in WB)? `P2a_cycles` (2 pt)
- (b) [Autograded] How many cycles would a sequence of 100 independent instructions take? `P2b_cycles` (2 pt)
- (c) [Autograded] In cycle 5, which instruction (I1–I5) is in each stage? `P2c_IF` `P2c_ID` `P2c_EX` `P2c_MEM` `P2c_WB` (5 pt)

- (d) **[Autograded]** In cycle 6, the WB stage reads the MEM/WB pipeline register. Which instruction's values does MEM/WB hold, and what register number does it send to the register file's *Write register* input? `P2d_instr` `P2d_rd` (2 pt)
- (e) **[Autograded]** In cycle 5, the EX stage reads the ID/EX pipeline register. What are the values of the MemWrite and RegWrite control bits that ID/EX carries in that cycle? `P2e_MemWrite` `P2e_RegWrite` (1 pt)

Problem 3: Data hazards and forwarding

(30 pt)

```
I1: lw    x5, 0(x10)
I2: add   x6, x5, x7
I3: sub   x8, x6, x5
I4: sw    x8, 4(x10)
I5: and   x9, x11, x12
I6: or    x13, x9, x6
```

- (a) **[Autograded]** List every read-after-write (RAW) dependency in this code: every case where an instruction reads a register that an earlier instruction in the sequence writes. Write each one as *producer-consumer:register*, separated by commas, for example `I7-I9:x20`. Include dependencies whether or not they cause a hazard. You earn 1 point for each correct entry and lose 1 point for each incorrect entry (minimum 0). `P3a_deps` (6 pt)
- (b) **[Autograded]** With **no forwarding**, how many stall cycles are needed, and how many total cycles does the sequence take? `P3b_stalls` `P3b_cycles` (4 pt)
- (c) **[Autograded]** With **full forwarding**, how many stall cycles are needed, and how many total cycles does the sequence take? `P3c_stalls` `P3c_cycles` (4 pt)
- (d) **[Autograded]** With full forwarding (including any stall from part (c)), where does each instruction get each source operand when it is in EX? Answer RF (the value read from the register file in ID, no forwarding), EXMEM (forwarded from the EX/MEM register), or MEMWB (forwarded from the MEM/WB register). For `sw`, `rs1` is the base register and `rs2` is the data register. `P3d_I2_rs1` `P3d_I2_rs2` `P3d_I3_rs1` `P3d_I3_rs2` `P3d_I4_rs1` `P3d_I4_rs2` `P3d_I5_rs1` `P3d_I5_rs2` `P3d_I6_rs1` `P3d_I6_rs2` (10 pt)
- (e) **[Written]** Draw the multi-cycle pipeline diagram for the **full forwarding** case. Show any stall clearly, and draw an arrow for every forwarding path used, from the cycle and stage where the value is available to the cycle and stage where it is used. You may fill in the grid below. (6 pt)

Instruction	1	2	3	4	5	6	7	8	9	10	11	12
I1: lw x5, 0(x10)												
I2: add x6, x5, x7												
I3: sub x8, x6, x5												
I4: sw x8, 4(x10)												
I5: and x9, x11, x12												
I6: or x13, x9, x6												

Problem 4: Code scheduling

(20 pt)

This C code, with A in x10 and B in x11 (both arrays of 4-byte ints),

```
A[0] = B[0] + B[1];
A[1] = B[2] - B[0];
A[2] = B[3] & B[1];
```

compiles to the following (this is also the starting contents of `schedule.s`):

```
lw x5, 0(x11)    # B[0]
lw x6, 4(x11)    # B[1]
add x7, x5, x6
sw x7, 0(x10)    # A[0]
lw x8, 8(x11)    # B[2]
sub x9, x8, x5
sw x9, 4(x10)    # A[1]
lw x12, 12(x11)  # B[3]
and x13, x12, x6
sw x13, 8(x10)   # A[2]
```

Assume **full forwarding** throughout this problem.

- (a) **[Autograded]** How many stall cycles does the original code have, and how many total cycles does it take? `P4a_stalls` `P4a_cycles` (4 pt)
- (b) **[Written]** Part (c) asks you to move loads of B above stores to A. What must be true about A and B for that to be legal? Give a specific example of A and B for which the reordered code would store different values than the original code. (4 pt)
- (c) **[Autograded]** Reorder the code to remove as many stalls as possible, and submit it as `schedule.s`. Rules:
- Use only `lw`, `sw`, `add`, `sub`, `and`, `or`. No labels.
 - Your code must leave the same values in `A[0]–A[2]` as the original code, must not modify B, and must not access memory outside `A[0..3]` and `B[0..3]`.
 - You may rename registers. Do not assume any register other than `x10` and `x11` holds a known value when the code starts.

The autograder runs your code on several inputs (with A and B in separate, non-overlapping memory) and then counts cycles with the forwarding model above. Incorrect code earns 0. Correct code earns 12 points at 14 cycles or fewer, 8 at 15, 4 at 16, and 0 at 17 or more. You see your result and a cycle-by-cycle timing table as soon as you submit. (12 pt)

Problem 5: Control hazards and branch prediction (18 pt)

In a program, 20% of instructions are branches, and 60% of those branches are taken. Without branch stalls the CPI would be 1.

- (a) **[Autograded]** Branches are resolved in ID, so a branch penalty is 1 cycle. What is the CPI if the pipeline always stalls on every branch? `P5a_cpi` (2 pt)
- (b) **[Autograded]** Same pipeline, but it predicts every branch not taken and pays the 1-cycle penalty only when a branch is taken. What is the CPI? `P5b_cpi` (2 pt)
- (c) **[Autograded]** A deeper pipeline resolves branches in MEM, so a penalty is 3 cycles. Give the CPI for: always stall; predict not-taken; and a dynamic predictor that is right 90% of the time (penalty only on a misprediction; a correct prediction, taken or not, costs nothing). `P5c_stall` `P5c_pnt` `P5c_pred` (6 pt)
- (d) **[Autograded]** In the deeper pipeline, what is the speedup of the dynamic predictor over always stalling? `P5d_speedup` (2 pt)
- (e) **[Autograded]** The loop below runs with $x5 = 0$, $x6 = 10$, $x7 = 1$, and $x11 = 4$. Count every execution of B1 and B2 from entering the loop until reaching done. Give the total number of branch executions, the number of mispredictions with *predict not-taken*, the number of mispredictions with static *backward-taken, forward-not-taken* (BTFN) prediction, and the BTFN accuracy as a percentage. `P5e_branches` `P5e_mispred_pnt` `P5e_mispred_btfn` `P5e_acc_btfn` (4 pt)

```

loop: beq  x5, x6, done      # B1
      lw   x8, 0(x10)
      add  x9, x9, x8
      add  x10, x10, x11
      add  x5, x5, x7
      beq  x0, x0, loop      # B2
done: ...

```

- (f) **[Written]** Using your numbers from (a)–(d), explain in 2–3 sentences why branch prediction matters more as pipelines get deeper. (2 pt)

Graduate Problem: Designing the forwarding unit [EECE.5821 only] (20 pt)

The forwarding unit in EX controls two 3-input multiplexers in front of the ALU. ForwardA selects the first ALU operand (for rs1) and ForwardB selects the second (for rs2):

Value	ALU operand comes from
00	ID/EX: the value read from the register file
10	EX/MEM: the ALU result of the previous instruction
01	MEM/WB: the value being written back (ALU result or loaded data)

The unit can see these fields: ID/EX.RegisterRs1, ID/EX.RegisterRs2, EX/MEM.RegWrite, EX/MEM.RegisterRd, MEM/WB.RegWrite, and MEM/WB.RegisterRd.

- (a) **[Written]** Write the complete Boolean conditions that set ForwardA to 10 and to 01. (ForwardB is the same with rs2.) Your conditions must handle register x0, instructions that do not write a register, and the case where both EX/MEM and MEM/WB hold a value for the same register. (5 pt)
- (b) **[Autograded]** Give ForwardA and ForwardB for each situation. In S2, the instruction in EX/MEM is a sw; its “rd” field contains bits of the store offset. (8 pt)

	EX/MEM.RegWrite	EX/MEM.Rd	MEM/WB.RegWrite	MEM/WB.Rd	ID/EX.Rs1	ID/EX.Rs2
S1	1	5	1	5	5	6
S2	0	7	1	7	7	7
S3	1	0	1	0	0	3
S4	1	9	1	4	4	9

G2_S1_ForwardA G2_S1_ForwardB G2_S2_ForwardA G2_S2_ForwardB
 G2_S3_ForwardA G2_S3_ForwardB G2_S4_ForwardA G2_S4_ForwardB (8 pt)

- (c) **[Autograded]** The lecture’s hazard detection unit in ID stalls when ID/EX.MemRead is 1 and ID/EX.RegisterRd equals either IF/ID.RegisterRs1 or IF/ID.RegisterRs2. Does it stall in each case below? Answer yes or no.

- L1: lw x5, 0(x4) is in EX; add x7, x5, x6 is in ID.
- L2: add x5, x1, x2 is in EX; sub x7, x5, x6 is in ID.
- L3: lw x8, 0(x4) is in EX; sw x8, 0(x9) is in ID.

G3_L1_stall G3_L2_stall G3_L3_stall (3 pt)

- (d) **[Written]** In case L3, the sw does not need x8 until its MEM stage. Describe a new forwarding path that would remove this stall: where it goes from and to, what condition enables it, and what extra information the pipeline registers must carry. (2 pt)
- (e) **[Written]** Explain why the condition for 01 must exclude the case where the 10 condition is true. Give a three-instruction example where the wrong value would be used otherwise. (2 pt)